

Network Administration / System Administration

Homework 1

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1. Discussion

- 這次作業我和 賴泓安、鄭竣陽 以及 喻慈恩 討論。

2. Reference

Problem 1. 參數檢查

- [How can I have a newline in a string in sh? \(Stack Overflow\)](#)
- [How can I match a string with a regex in Bash? \(Stack Overflow\)](#)
- [正規表達式 - wikipedia](#)
- [How do I use a regex in a shell script? \(Stack Overflow\)](#)
- [Regular Expression 正則表達式初探 \(TimCodingBlog\)](#)
- [test man page](#)

Problem 2. Building a Merkle Tree

- [How can I store the "find" command results as an array in Bash \(Stack Overflow\)](#)
- [Add a new element to an array without specifying the index in Bash \(Stack Overflow\)](#)
- [Loop through an array of strings in Bash? \(Stack Overflow\)](#)
- [Convert absolute path into relative path given a current directory using Bash \(Stack Overflow\)](#)
- [test man page](#)

Problem 3. Generating an Inclusion Proof

- [Bash bitwise operators - xor \(Sysxplore\)](#)

Problem 4. Verifying an Inclusion Proof

- [Using multiple delimiters in awk \(Stack Overflow\)](#)
- [Bash bitwise operators - and \(Sysxplore\)](#)
- [How to convert a string to lower case in Bash \(Stack Overflow\)](#)

3. AI conversation

Problem 1. 參數檢查

幫我確認我 regex 有沒有寫錯

```
if [[ ! "$root" =~ ^[A-F0-9][a-f0-9]$ ]]; then
```

你的正則表達式有一個邏輯錯誤，導致它的行為可能不是你預期的。讓我們來分析它：

問題點

1. `^[A-F0-9][a-f0-9]$`

- `^` 代表匹配開頭，`$` 代表匹配結尾。
- 但 `|` (或運算) 將它拆成兩部分：
 - `^[A-F0-9]`: 匹配以 **A-F 或 0-9** 開頭的字串
 - `[a-f0-9]$`: 匹配以 **a-f 或 0-9** 結尾的字串
- 這樣的話，它會錯誤地匹配：
 - 任何以 **A-F 或 0-9** 開頭的字串 (不論長度)
 - 任何以 **a-f 或 0-9** 結尾的字串 (不論長度)
- 這表示 **單個字元符合條件時，整個條件就會通過**，這可能不是你的本意。

修正方式

如果你的目標是：

- 確保 `root` 只包含 16 進位字元 (A-F、a-f、0-9)，那麼該這樣寫：

bash

複製

```
if [[ ! "$root" =~ ^[A-Fa-f0-9]+$ ]]; then
```

解釋：

- `^` 開頭
- `[A-Fa-f0-9]+` 表示 **至少一個 16 進位字元**
- `$` 結尾
- 整體表示：「整個字串都必須是 16 進位字元」

這樣才是完整匹配整個字串，而不只是開頭或結尾。

錯誤示範 vs. 修正後

錯誤寫法

bash

複製

```
[[ ! "$root" =~ ^[A-F0-9][a-f0-9]$ ]]
```

- ✓ `Axyz` → 通過 (因為 A 開頭)
- ✓ `xyz3` → 通過 (因為 3 結尾)
- ✗ `xyzG` → 不符合 16 進位字元，但錯誤寫法仍然可能讓它錯誤通過

正確寫法

bash

複製

```
[[ ! "$root" =~ ^[A-Fa-f0-9]+$ ]]
```

- ✓ `A3F9b` → 符合
- ✓ `123abc` → 符合
- ✗ `xyzG` → 正確排除 (因為 G 不是 16 進位字元)
- ✗ `hello123` → 正確排除 (因為 h 不是 16 進位字元)

這樣就能準確檢查 `root` 是否完全由 16 進位字元組成！

只含有數字 0 至 9 及字母 A 到 F，使用的字母為全大寫或全小寫。

如果你的需求是「只含有 0-9 及 A-F，且字母要全大寫或全小寫」，那我們需要：

- 1. 確保 只含有 0-9 和 A-F。
- 2. 確保 全部是大寫或全部是小寫。

修正後的 Bash 判斷式

bash

複製

```
if [[ ! "$root" =~ ^([A-F0-9]+|[a-f0-9]+)$ ]]; then
```

解釋

- `^([A-F0-9]+|[a-f0-9]+)$`
 - `^`: 字串開頭
 - `([A-F0-9]+|[a-f0-9]+)`:
 - `[A-F0-9]+` → 全大寫數字
 - `|` → 或
 - `[a-f0-9]+` → 全小寫數字
 - `$`: 字串結尾
- 這確保：
 - 字串內的 字母全為大寫或全為小寫。
 - 只能包含 0-9 及 A-F/a-f。

測試案例

測試字串	期望結果	為何符合 / 不符合
1234ABCD	✔ 符合	全部大寫數字
abcd1234	✔ 符合	全部小寫數字
A1b2C3	✖ 不符合	有大小寫混合
G1234	✖ 不符合	包含 G，不是 0-9, A-F
12345	✔ 符合	只有數字，符合條件
AaBbCc	✖ 不符合	混合大小寫
abcdef	✔ 符合	全小寫 A-F

這樣就能確保 root 只包含 大寫或小寫的 16 進位字元，而且不會混合大小寫！



Problem 2. Building a Merkle Tree

find path/to/dir | LC_COLLATE=C sort 的結果要如何存到 array

在 Bash 中，你可以使用命令替換（`$(...)`）將 `find` 和 `sort` 的結果存入一個 array。例如：

```
bash
mapfile -t files < <(find path/to/dir | LC_COLLATE=C sort)
```

解釋：

1. `find path/to/dir`：尋找 `path/to/dir` 內的所有檔案與目錄。
2. `LC_COLLATE=C sort`：使用 `C` 語系來排序，確保排序結果是按 byte 值排序（類似 `strcmp` 的行為）。
3. `< <(...)`：這是 **process substitution**，確保 `mapfile` 能夠讀取命令輸出的每一行並存入 array。
4. `mapfile -t files`：將結果讀入 `files` 陣列，`-t` 會去掉結尾的換行符。

訪問陣列內容：

- 顯示陣列內容：

```
bash
printf '%s\n' "${files[@]}"
```

- 取得第一個元素：

```
bash
echo "${files[0]}"
```

- 迴圈遍歷：

```
bash
for file in "${files[@]}; do
    echo "$file"
done
```

這樣你就能把 `find` 和 `sort` 的結果存入陣列並方便地操作它了！🚀

📄 🍌 🗑 🗣 🔄

Problem 3. Generating an Inclusion Proof

我要怎麼一行一行讀取一個檔案直到空行，然後改為將接下來每一行的資料一行一行按照：分開使用 bash

你可以使用 `while read` 來逐行讀取檔案，並在遇到空行 ("") 後，將接下來的每一行按照：分割。

解法：

```
bash
#!/bin/bash

file="your_file.txt"
reading_first_part=true # 用來標記是否在第一部分

while IFS= read -r line; do
    if [ -z "$line" ]; then
        reading_first_part=false
        continue # 跳過空行
    fi

    if $reading_first_part; then
        echo "First part: $line"
    else
        IFS=":" read -r first second <<< "$line"
        echo "First: $first, Second: $second"
    fi
done < "$file"
```

說明：

1. `reading_first_part=true` 來標記是否仍在第一部分。
2. 遇到空行 (`[ -z "$line" ]`) 時，切換 `reading_first_part=false`，並跳過該行。
3. 第一部分的行直接輸出。
4. 第二部分的行用 `IFS=":" read -r first second <<< "$line"` 來分割並輸出。

這樣就能一行一行讀取檔案，直到空行，然後對接下來的行按照：分割。 🚀

📄 👍 💬 🔊

第二部分可能有未知個

如果第二部分的每一行可能有未知數量的：分隔的欄位，你可以使用 `IFS=":" read -ra parts` 來將整行切割成一個陣列。這樣就能動態處理欄位數量。

Bash Script：

```
bash
#!/bin/bash

file="your_file.txt"
reading_first_part=true # 標記是否在第一部分

while IFS= read -r line; do
    if [ -z "$line" ]; then
        reading_first_part=false
        continue # 跳過空行
    fi

    if $reading_first_part; then
        echo "First part: $line"
    else
        IFS=":" read -ra parts <<< "$line"
        echo "Second part fields:"
        for field in "${parts[@]}; do
            echo " - $field"
        done
    fi
done < "$file"
```

說明：

1. `IFS=":" read -ra parts <<< "$line"` 會將 `line` 按：分割成陣列 `parts`。
2. 用 `for field in "${parts[@]}"` 來逐一輸出陣列內的所有欄位。
3. 遇到空行後才開始處理：分割的內容。

Problem 4. Verifying an Inclusion Proof

我要怎麼讀取

leaf_index:9,tree_size:10

當中的數字呢？

使用 bash

你可以使用 `awk` 或 `grep + cut` 來提取 `leaf_index` 和 `tree_size` 的數值。以下是幾種方法：

方法 1：使用 `awk`

```
bash
echo "leaf_index:9,tree_size:10" | awk -F'[:,' '{print "leaf_index="$2, "tree_size="$4}'
```

輸出：

```
leaf_index=9 tree_size=10
```

如果你只想要某一個數值，例如 `leaf_index`：

```
bash
echo "leaf_index:9,tree_size:10" | awk -F'[:,' '{print $2}'
```

輸出：

```
9
```

方法 2：使用 `grep + cut`

```
bash
leaf_index=$(echo "leaf_index:9,tree_size:10" | grep -oP 'leaf_index:\K\d+')
tree_size=$(echo "leaf_index:9,tree_size:10" | grep -oP 'tree_size:\K\d+')
echo "leaf_index=$leaf_index"
echo "tree_size=$tree_size"
```

輸出：

```
leaf_index=9
tree_size=10
```

方法 3：使用 `sed`

```
bash
leaf_index=$(echo "leaf_index:9,tree_size:10" | sed -n 's/.*leaf_index:([0-9]*).*$/\1/p')
tree_size=$(echo "leaf_index:9,tree_size:10" | sed -n 's/.*tree_size:([0-9]*).*$/\1/p')
echo "leaf_index=$leaf_index"
echo "tree_size=$tree_size"
```

這些方法都可以提取數值，你可以根據需求選擇適合的方式！